

Object oriented Programming | SAQ

1. Define object with example

An object is an instance of a class. It represents a real-world entity and contains data (variables) and behavior (methods).

```
class Student {  
    String name;  
  
    Student(String n) {  
        name = n;  
    }  
  
    void display() {  
        System.out.println("Name: " + name);  
    }  
  
    public static void main(String[] args) {  
        Student s1 = new Student("Rahim");  
        s1.display();  
    }  
}
```

Here, `s1` is an object of the Student class.

2. Suppose you have three variables: One for temporary calculation, one for storing object-specific data, and one shared across all objects. Identify which variable should be local, instance, and static.

Java variables can be classified into local, instance, and static variables based on their use.

1. **Local Variable:** A variable declared inside a method or block. It is used for temporary calculations.
2. **Instance Variable:** A variable declared inside a class but outside methods. It stores object-specific data.
3. **Static Variable:** A variable declared using `static`. It is shared by all objects of the class.

Example:

```
class Student {  
    static int count; // Static variable  
    String name; // Instance variable  
  
    void display() {  
        int x = 10; // Local variable  
    }  
}
```

- Temporary calculation → Local variable
- Object-specific data → Instance variable
- Shared by all objects → Static variable

4. You need to illustrate a method that should be accessible to subclasses but not accessible to unrelated classes in other packages. Choose the most appropriate access modifier for this method.

protected Access Modifier

The most appropriate access modifier is `protected`.

- A protected method can be accessed by subclasses, even when they are in a different package.
- It cannot be directly accessed by unrelated classes in other packages.

Example:

```
protected void display() {  
    System.out.println("Hello");  
}
```

Therefore, `protected` is the correct access modifier.

5. Evaluate how access modifier control the visibility of class members, and provide one example to support your explanation.

Access Modifier and Visibility

An access modifier controls the visibility and accessibility of class members such as variables and methods.

Java has four access modifiers:

- `private` → accessible only within the same class.
- `default` → accessible within the same package.

- `protected` → accessible within the same package and subclasses.
- `public` → accessible from anywhere.

Example

```
class Student {  
    private int age = 20;  
    public String name = "Rahim";  
}
```

Here, `age` is private, so it can only be accessed inside the Student class. `name` is public, so it can be accessed from anywhere. Thus, access modifiers control the visibility of class members.

6. Demonstrate the concept of polymorphism in Java with an example.

Polymorphism means “many forms.” In Java, it allows the same method name to perform different tasks depending on the object or parameters.

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    public static void main(String[] args) {  
        Calculator c = new Calculator();  
        System.out.println(c.add(10, 20));  
        System.out.println(c.add(10, 20, 30));  
    }  
}
```

Java Program: Calculate Average of Three Marks

```
import java.util.Scanner;

public class Average {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter marks of three subjects: ");
        double mark1 = sc.nextDouble();
        double mark2 = sc.nextDouble();
        double mark3 = sc.nextDouble();

        double average = (mark1 + mark2 + mark3) / 3;

        System.out.println("Average = " + average);
    }
}
```

Example Output

```
Enter marks of three subjects: 80 70 90
Average = 80.0
```

7. Find the states and behaviours of car object.

States and Behaviors of a Car Object

States (Attributes):

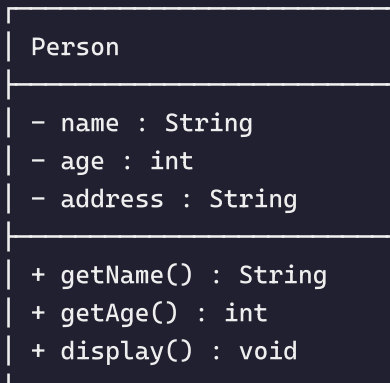
- Color
- Brand
- Model
- Speed

Behaviors (Methods):

- Start()
- Stop()
- Accelerate()
- Brake()

8. Construct a UML Class Diagram for person class.

UML Class Diagram for Person Class



Note: `-` means private and `+` means public.

9. Write the syntax of creating class and object

Steps of Creating Class and Object

Creating a Class

```
class Student {
    String name;

    void display() {
        System.out.println(name);
    }
}
```

Creating an Object

```
Student s1 = new Student();
```

10. Show details objects of university and find their inter-connection.

Object Modeling

Object modeling means identifying the objects of a system, describing their properties and behaviors, and finding the relationships between them.

Example: University System

Objects:

- Student
- Teacher
- Course
- Department

Interconnections:

- A Student enrolls in a Course.
- A Teacher teaches a Course.
- A Department offers Courses.
- A Student belongs to a Department.

In short: Object modeling shows the objects of a system and how they are interconnected.

Method Overloading

```
void display(int a)
void display(double a)
```

a) Analyze the above methods and state whether they are overloaded or not.

b) What happens if two methods have the same name and the same parameter list?

A. Yes, the methods are overloaded because they have the same method name `display()` but different parameter types (`int` and `double`).

```
void display(int a)
void display(double a)
```

This is an example of method overloading.

b) If two methods have the same name and same parameter list, they are not considered overloaded.

Java does not allow two methods with the same signature in the same class.

It creates a compile-time error.

The compiler cannot distinguish between the two methods.

For method overloading, the parameter list must be different.

The return type alone cannot be used for method overloading.

1. Define class

Class in Java

A class in Java is a blueprint or template for creating objects.

It defines the data (variables) and behaviors (methods) of objects.

A class is declared using the `class` keyword.

Example:

```
class Student {
    String name;

    void display() {
        System.out.println(name);
    }
}
```

2. A program has three variables: one declared inside a method, one declared inside a class without static, and one declared with static. Identify which is local, which is instance, and which is static. Justify your answer.

1. **Local Variable:** The variable declared inside a method is a local variable because its scope is limited to that method.
2. **Instance Variable:** The variable declared inside a class without `static` is an instance variable because each object has its own copy.
3. **Static Variable:** The variable declared with `static` is a static variable because it is shared by all objects of the class.

Example:

```
class Student {
    int age; // Instance variable
    static int count; // Static variable

    void display() {
        int marks = 80; // Local variable
    }
}
```

4. State the meaning of an access modifier in Java and give one example

An access modifier in Java is a keyword that controls the visibility and accessibility of classes, variables, methods, and constructors.

Example:

```
private int age;
```

Here, `private` is an access modifier, so `age` can be accessed only within the same class.

5. Which access modifier would you choose for a method that should be accessible by subclasses but not by unrelated classes in Other packages? Explain your choice.

The most appropriate access modifier is `protected`.

- A protected method can be accessed by subclasses, even if they are in another package.
- It cannot be directly accessed by unrelated classes in other packages.
- Therefore, `protected` provides the required level of access.

Example:

```
protected void display() {  
    System.out.println("Hello");  
}
```

Therefore, the correct choice is `protected`.

6. Design a Java program using the Scanner class that accepts three subject marks from the user, calculates the average, and displays the result

Java Program Using Scanner

```
import java.util.Scanner;

public class Average {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter marks of three subjects: ");
        int m1 = sc.nextInt();
        int m2 = sc.nextInt();
        int m3 = sc.nextInt();

        double average = (m1 + m2 + m3) / 3.0;

        System.out.println("Average = " + average);
    }
}
```

Example Output

```
Enter marks of three subjects: 80 70 90
Average = 80.0
```

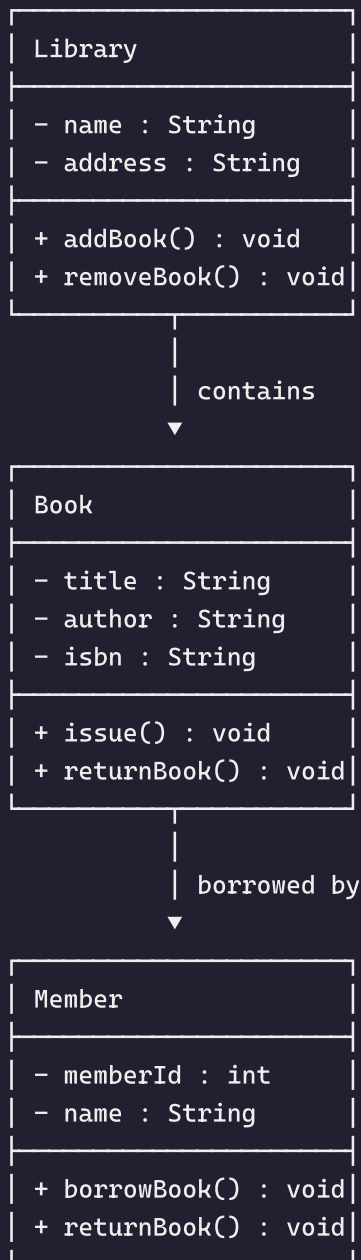
7. Define UML

UML (Unified Modeling Language) is a standard visual language used to design and represent the structure and behavior of a software system.

It uses diagrams to show classes, objects, relationships, and interactions in a system.

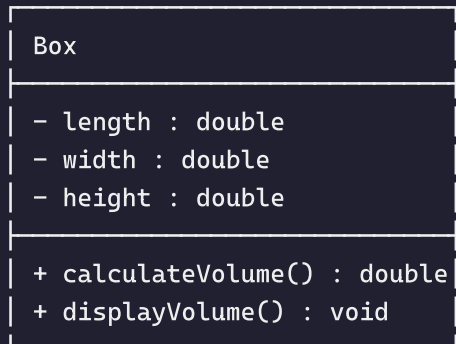
8. Construct a UML Class Diagram for the Library Management System.

UML Class Diagram: Library Management System



Relationship: A Library contains Book objects, and a Member can borrow and return books.

9. Apply UML notation to model a Box class with attributes (length, width, height) and an operation for calculating and displaying the volume



Volume = length × width × height

-  → Private
-  → Public
- **Attributes:** length, width, height
- **Operations:** calculateVolume() and displayVolume()

Consider the following UML Diagram:

a) Convert the above UML Diagram into Java code.

b) Create two objects of the Department class, set the values, and display the information using Java language

a) Java Code

```
public class Department {
    String deptName;
    int deptCode;
    String faculty;

    public static void main(String[] args) {
    }
}
```

b) Create Two Objects, Set Values and Display Information

```
public class Department {  
    String deptName;  
    int deptCode;  
    String faculty;  
  
    public static void main(String[] args) {  
        Department d1 = new Department();  
        d1.deptName = "Computer Science";  
        d1.deptCode = 101;  
        d1.faculty = "Science";  
  
        Department d2 = new Department();  
        d2.deptName = "English";  
        d2.deptCode = 102;  
        d2.faculty = "Arts";  
  
        System.out.println("Department 1:");  
        System.out.println(d1.deptName);  
        System.out.println(d1.deptCode);  
        System.out.println(d1.faculty);  
  
        System.out.println("\nDepartment 2:");  
        System.out.println(d2.deptName);  
        System.out.println(d2.deptCode);  
        System.out.println(d2.faculty);  
    }  
}
```

Output

```
Department 1:  
Computer Science  
101  
Science  
Department 2:  
English  
102  
Arts
```